- **MIT-BIH Arrhythmia** is a cleaner, benchmark-style ECG dataset with beat annotations.
- **MIMIC-III** is a real clinical database with messy, heterogeneous, real-world data such as demographics, diagnoses, labs, ICU events, and sometimes waveform signals depending on which MIMIC resource you use.

So the preprocessing should be done in **four layers**:

1. Define what your agentic AI will decide

Before preprocessing, define the task clearly. For example:

- arrhythmia classification
- cardiac risk prediction
- abnormal beat detection
- ICU deterioration prediction for heart patients
- recommendation support for diagnosis or triage

A practical design is:

- **MIT-BIH** for ECG morphology and beat-level arrhythmia learning
- **MIMIC-III** for patient context, labs, vitals, comorbidities, medications, and outcomes

That means your agent can combine:

- ECG pattern understanding
- clinical reasoning
- temporal patient context
- rule-based or knowledge-guided decision support

2. Preprocess MIT-BIH Arrhythmia Database

MIT-BIH is usually used for ECG signal preprocessing and supervised classification.

Step 2.1 Load ECG signals and annotations

Each record contains:

- two-channel ECG waveform
- sampling frequency usually 360 Hz
- annotation files for beat locations and labels

You need to extract:

- raw ECG signal
- R-peak positions
- beat labels

Step 2.2 Remove noise

Common ECG denoising steps:

- remove baseline wander
- remove powerline interference
- remove muscle noise

Typical methods:

- bandpass filter, often around 0.5 to 40 Hz
- notch filter at 50 or 60 Hz
- wavelet denoising if needed

Step 2.3 Normalize signal

Normalize per record or per segment:

- z-score normalization
- min-max normalization

Z-score is common for deep learning because it stabilizes amplitude differences.

Step 2.4 Detect or verify R-peaks

Even though MIT-BIH has annotations, it is useful to verify beat alignment.

You can use:

- annotated R-peaks directly for supervised learning
- or run your own detector for robustness experiments

Step 2.5 Segment beats or windows

Two common strategies:

Beat-level segmentation

Cut around each R-peak, for example:

- 100 samples before R-peak
- 150 or 200 samples after R-peak

This gives one heartbeat segment.

Window-level segmentation

Use fixed-length windows such as:

- 5 seconds
- 10 seconds

This is better if your model needs rhythm context rather than only one beat.

Step 2.6 Map annotation labels

MIT-BIH has many beat symbols. Usually you map them into larger classes, such as **AAMI 5 classes**:

- N: normal
- S: supraventricular ectopic
- V: ventricular ectopic
- F: fusion
- Q: unknown or unclassifiable

This is important because raw labels are too fragmented.

Step 2.7 Handle class imbalance

MIT-BIH is heavily imbalanced. Normal beats dominate.

Possible solutions:

- undersampling normal beats
- oversampling minority classes
- SMOTE on extracted features
- class-weighted loss
- focal loss

For ECG deep learning, class-weighted loss or focal loss is often better than aggressive oversampling.

Step 2.8 Split data correctly

Do not randomly split beats across train and test.

Use **record-wise split** or **subject-independent split**, because beat-wise random split causes leakage.

This is very important.

Step 2.9 Optional feature extraction

You can use either:

Raw signal input

Feed the segmented waveform directly into CNN, Transformer, or hybrid models.

Handcrafted features

Extract:

- RR interval
- QRS duration
- P-wave features
- T-wave features
- heart rate variability
- waveform morphology
- wavelet coefficients

For agentic AI, handcrafted features are valuable because they are interpretable and can support reasoning rules.

3. Preprocess MIMIC-III

MIMIC-III is more complex. You first need to decide which part you want:

- structured clinical tables only
- waveform data
- both

If your goal is cardiac decision support, a very good setup is:

- use **structured MIMIC-III tables** for patient context
- use ECG or waveform-related data when available
- align outcomes such as mortality, arrhythmia-related events, ICU deterioration, or diagnosis groups

Step 3.1 Select target cohort

Build a cardiac-related cohort, for example patients with:

- heart failure
- myocardial infarction
- arrhythmia
- coronary artery disease
- cardiac ICU admissions

You can identify them using:

- ICD-9 diagnosis codes
- admission diagnoses
- procedure codes
- cardiac medications
- lab markers like troponin when available

Step 3.2 Extract relevant tables

Important MIMIC-III tables often include:

- PATIENTS
- ADMISSIONS
- ICUSTAYS
- DIAGNOSES_ICD
- PROCEDURES_ICD
- PRESCRIPTIONS
- LABEVENTS
- CHARTEVENTS
- INPUTEVENTS
- OUTPUTEVENTS
- NOTEEVENTS if you want text

For cardiac tasks, especially useful variables are:

- age

- sex
- admission type
- diagnosis history
- blood pressure
- heart rate
- SpO2
- respiratory rate
- troponin
- CK-MB
- electrolytes
- creatinine
- BNP if available
- medications
- interventions
- mortality or ICU outcome

Step 3.3 Clean identifiers and timestamps

You need to join tables by:

- SUBJECT_ID
- HADM_ID
- ICUSTAY_ID

Then standardize time:

- admission time
- ICU start time
- event chart time
- lab time

Time alignment is critical because MIMIC is longitudinal.

Step 3.4 Handle missing data

MIMIC has lots of missingness.

Options:

- remove variables with too much missing data
- forward fill for time series in short windows
- mean or median imputation for stable variables

- interpolation for continuous monitoring data
- missing indicator flags

Important: for clinical AI, do not hide missingness completely. Sometimes missingness itself carries meaning.

Step 3.5 Remove outliers and unit inconsistencies

MIMIC variables may contain:

- impossible values
- duplicate entries
- charting errors
- unit mismatches

Examples:

- heart rate too large
- temperature recorded in different units
- lab values with inconsistent scales

You should apply physiologic range checks and convert everything to standard units.

Step 3.6 Convert categorical variables

For variables like:

- admission type
- ethnicity
- medication category
- diagnosis category

Use:

- one-hot encoding
- embedding layers
- grouped clinical categories

For agentic AI, grouped categories are often more interpretable than overly sparse encoding.

Step 3.7 Aggregate temporal data

MIMIC includes irregular time-series data. You need to convert it into usable windows.

Examples:

- first 24 hours after ICU admission
- every 1-hour window
- every 6-hour window

For each window, calculate:

- mean
- min
- max
- trend
- last observed value
- variability

This converts irregular chart events into model-ready features.

Step 3.8 Define labels or outcomes

Possible labels:

- in-hospital mortality
- arrhythmia event
- cardiac arrest risk
- ICU deterioration
- readmission
- diagnosis category

Your preprocessing depends on the label definition. For example, if predicting deterioration within 6 hours, only use data before that horizon.

Step 3.9 Normalize numerical features

Use:

- z-score normalization
- robust scaling for skewed labs
- log transform for highly skewed variables

Do this using training data only, then apply the same statistics to validation and test sets.

4. If using MIMIC waveform or ECG signals

If you use waveform ECG from the MIMIC ecosystem, preprocess it similarly to MIT-BIH, but it is messier.

Steps:

- resample signals if sampling frequencies differ
- filter noise
- detect signal quality
- discard corrupted segments
- segment fixed windows or heartbeats
- align waveform timestamps with ICU events and diagnoses

A very important extra step here is **signal quality indexing** because ICU waveforms often contain artifacts.

5. Align MIT-BIH and MIMIC into one framework

This is the most important part for your agentic AI.

Because these datasets are different, you usually do not merge them row by row. Instead, you use them in complementary roles.

Option A: Pretrain and transfer

- train ECG encoder on MIT-BIH for arrhythmia morphology
- transfer that encoder to MIMIC ECG or waveform tasks
- combine encoder outputs with structured MIMIC clinical features

This is a strong strategy.

Option B: Multi-branch model

Build two branches:

- ECG branch from MIT-BIH-style or MIMIC waveform input
- clinical branch from MIMIC structured data

Then fuse:

- feature concatenation
- attention-based fusion

- gated fusion

Option C: Agentic pipeline

Create multiple agents/modules:

- **signal agent**: analyzes ECG morphology and rhythm
- **clinical agent**: analyzes labs, vitals, medications, history
- **knowledge agent**: applies cardiology rules
- **decision agent**: combines outputs and produces recommendation

This is especially good if you want explainable AI.